

ARQUITECTURA DE COMPUTADORES

Proyecto 3

Grupo 15:

Integrantes:

Martin Heiremans

Felix Svensson

Juan Pablo Saavedra

Profesor:

Jorge Gómez Mir

Santiago, 4 de Noviembre de 2026

ÍNDICE

1. INTRODUCCIÓN **1**
2. DISEÑO GENERAL Y ARQUITECTURA **3**
Figura 1: Diagrama de bloques del sistema 3
2.1 Modificaciones al RTL 4
3. TEMPORIZACIÓN Y MEDICIÓN DEL TIEMPO **5**
3.1 Frecuencia de reloj y unidad de tiempo 5
3.2 Contador de ciclos y espera de tres segundos 5
3.3 Medición del tiempo de reacción 5
3.4 Conversión de ciclos a décimas 6
3.5 Cálculo del promedio 6
3.6 Selección pseudoaleatoria del LED objetivo 6
4. PROGRAMA EN ASSEMBLY **7**
Figura 2: Diagrama de flujo del programa 7
5. ASSEMBLER **8**
5.1 Sintaxis e instrucciones 8
5.2 Etiquetas e inmediatos 8
5.3 Validación y pruebas 8
5.4 Ejemplo de traducción 9

1. DEL RTL DE FPGA AL RTL DE ASIC

El módulo superior asic/rtl/asic_top.v instancia el SoC sin elementos propios de la FPGA. Sus puertos son el reloj (i_Clk), el reset activo bajo (rst_ni), cuatro botones (i_Switch), cuatro LEDs (o_LED), dos displays de 7 segmentos (o_Segment1_A…G y o_Segment2_A…G) y el SPI (i_SPI_SCLK, i_SPI_MOSI, i_SPI_CS_n y o_SPI_MISO). Son terminales del macro, no pads físicos.

Las construcciones del RTL del Proyecto 2 que dependían de la FPGA se identificaron revisando el código. El RTL original no se pasó por LibreLane, así que no hay un mensaje del flujo asociado a cada una; la tabla explica por qué cada una es un problema en un ASIC y cómo se reemplazó.

Construcción	**Problema en ASIC**	**Reemplazo**
Reset de encendido con el contador por_cnt inicializado (rtl/pochoco_soc.v)	En silicio un flip-flop no parte con un valor conocido; el valor inicial no se fabrica.	Se eliminó; el reset llega por el terminal rst_ni.
initial y $readmemh(game.hex) en la memoria (rtl/pochoco_ram.v)	No hay un archivo que se cargue al encender; initial no se sintetiza.	A: ROM combinacional generada por el assembler. B: cada palabra toma su valor con el reset. Ambas mantienen la lectura síncrona que espera Espino.
SB_RAM40_4K en el register file (rtl/espino_core/espino_register_file.v)	Es una primitiva de la iCE40 que no existe en GF180.	15 registros de 32 bits con flip-flops y dos lecturas registradas.
Constantes de tiempo de game.s para el reloj de 25 MHz	El macro funciona con T = 100 ns (10 MHz).	Constantes recalculadas (sección 4).

2. MEMORIA Y PROGRAMA DEL JUEGO

El assembler (assembler/assembler.py, opción --rom-dir) genera desde sw/game.s la ROM combinacional de la estrategia A (program_rom.v) y los valores de reset de la estrategia B (program_reset.vh). A usa esa ROM más una RAM de 1 palabra; B es una memoria unificada de 108 palabras de 32 bits en flip-flops, cuyo contenido se carga con el reset.

El juego usa 107 palabras de instrucciones (428 bytes), que cuenta el assembler (program_meta.vh), y 0 palabras de datos: guarda su estado en registros y todos sus lw/sw van a los periféricos (base 0x80000000). La simulación lo confirma con 0 accesos a RAM. Se dejó 1 palabra de datos (0x1AC) para probar lectura y escritura. Ambas cantidades están bajo el máximo recomendado de 256 y 64 palabras.

Ambas estrategias se implementaron con T = 100 ns y 30 % de utilización, porque B no completó el flujo al 40 %. La frecuencia máxima es la del menor período que cumple timing en los 9 corners, medido con STA sobre cada layout.

Estrategia	**Área de celdas (µm²)**	**Flip-flops**	**Utilización final**	**Frecuencia máxima**
A: ROM + RAM de 1 palabra	154.474	869	39,2 %	17,54 MHz (57 ns)
B: memoria unificada de 108 palabras	628.066	4.293	41,5 %	10,99 MHz (91 ns)

B ocupa 4,1 veces el área de celdas de A y es más lenta, porque guarda el programa en flip-flops con sus multiplexores de lectura. Por eso la corrida final usa A.

3. SÍNTESIS

Los datos son del reporte de síntesis de la corrida final (results/runs/final_A_v2_p100_u40/synthesis.log). Con SYNTH_HIERARCHY_MODE = deferred_flatten, Yosys entrega las estadísticas por módulo antes de aplanar y se conservan las instancias u_rf (register file), u_alu (ALU) y u_cycle_counter (contador de ciclos). Las cinco celdas más usadas son:

Celda	Cantidad	Parte del diseño que la explica
dffrnq_1	866	Flip-flops con reset. 544 son del register file (15 × 32 bits más las dos salidas de lectura de 32 bits); el resto está en la memoria, los periféricos, el SPI y el contador (32).
mux2_2	736	480 son del register file: cada bit elige entre mantener su valor o escribir el dato nuevo.
nand2_1	698	Lógica repartida entre el register file, la ALU, la LSU, la decodificación y la ROM.
aoi22_1	539	384 son del register file: forman los multiplexores de sus dos puertos de lectura.
nor2_1	412	Lógica repartida entre los periféricos, la memoria, la ROM y la ALU.

El módulo con más área es el register file, con 71.587,67 μm², el 46,3 % de los 154.625,50 μm² del diseño antes de aplanar. La ALU ocupa 10.152,80 μm² (6,6 %) y el contador de ciclos 3.942,58 μm² (2,55 %).

Antes de sintetizar se esperaba que el register file fuera uno de los bloques más grandes, por sus 480 bits de estado y sus dos puertos de lectura (docs/PREDICCION_AREA.md). El resultado coincide: es el bloque más grande y explica la mayoría de los flip-flops, mux2 y aoi22.

4. TIMING

Se barrió CLOCK_PERIOD en 50, 60, 80 y 100 ns, con 40 % de utilización. La tabla muestra WNS / TNS en ns para el corner típico (TT, nom_tt_025C_5v00) y el más lento (SS, max_ss_125C_4v50).

CLOCK_PERIOD	Setup TT	Setup SS	Hold TT	Hold SS	Cierra
50 ns	0 / 0	-8,868 / -581,859	0 / 0	0 / 0	No
60 ns	0 / 0	0 / 0	0 / 0	0 / 0	Sí
80 ns	0 / 0	0 / 0	0 / 0	0 / 0	Sí
100 ns	0 / 0	0 / 0	0 / 0	0 / 0	Sí

El menor período que cierra timing en todos los corners es 60 ns, así que la frecuencia máxima es 16,67 MHz. Cada valor de CLOCK_PERIOD puede provocar una nueva optimización y cambiar el área, el buffering y el placement; en este barrido el área de celdas fue la misma (154.474 µm²) y solo cambió el slack.

La ruta crítica de la corrida final (SS, slack 41,46 ns) va de u_pochoco.u_ram._604_/Q (instrucción leída, instr_rdata[4]) a u_pochoco.u_ram._648_/D (dato leído, ram_b_rdata[16]), en el módulo asic_memory (u_ram), pasando por la decodificación y la LSU. Corresponde a acceso a memoria: en un ciclo se decodifica la instrucción, se calcula la dirección y se lee la ROM.

Para acortarla se registró la dirección del puerto de datos antes de leer la ROM (asic/variants/asic_memory_registered_addr.v); la variante pasa las mismas pruebas. El slack solo sube de 41,132 a 41,246 ns y el área de celdas sube 2,4 % (de 154.474 a 158.195 µm²), porque la ruta crítica pasa a otra de largo parecido (de instr_rdata[0] a led_cycle_q[20]). Por eso no se usó en la corrida final.

Con T = 100 ns (f = 10 MHz) cambian tres constantes de game.s, antes calculadas para 25 MHz: la espera de tres segundos pasa a 3 s × 10 MHz = 30.000.000 ciclos (antes 75.000.000); una décima de segundo, a 0,1 s × 10 MHz = 1.000.000 (antes 2.500.000); y el divisor del promedio de las diez rondas, a 10 × 1.000.000 = 10.000.000 (antes 25.000.000). El programa se regeneró con make -f Makefile.asic generate.

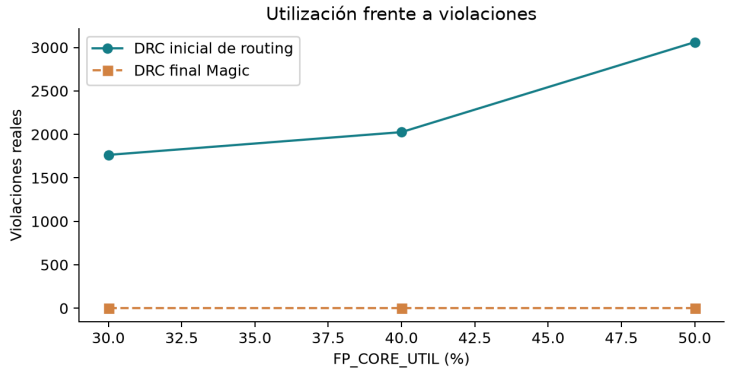
5. FLOORPLAN, RED DE ALIMENTACIÓN Y PLACEMENT

LibreLane genera el floorplan, la red de alimentación VDD/VSS y el placement para GF180MCU D. Se probaron tres valores de FP_CORE_UTIL con T = 100 ns, y los tres completan el flujo con DRC, LVS y antena limpios y cumplen timing.

FP_CORE_UTIL	Área del die (µm²)	Ocupación final	Costo equivalente
30 %	547.483	39,2 %	US$ 12.592
40 %	414.450	51,4 %	US$ 9.532
50 %	334.270	63,7 %	US$ 7.688

Entre los valores probados, el área mínima con la que el diseño completa el flujo es 334.270 µm² (50 %). Al subir la utilización aumentan las violaciones iniciales del router, aunque el DRC final es 0 en los tres casos (Figura 1).

Figura 1: Violaciones iniciales de routing y DRC final según la utilización



Los terminales se ubicaron por función (asic/constraints/pins.cfg): reloj y reset al norte, botones al oeste, LEDs y displays al este, y SPI al sur, para que cada interfaz quede en un solo borde y las entradas no se crucen con las salidas. Son pines de metal del macro, sin protección ESD ni capacidad para manejar cargas externas; un chip completo necesitaría pads de I/O.

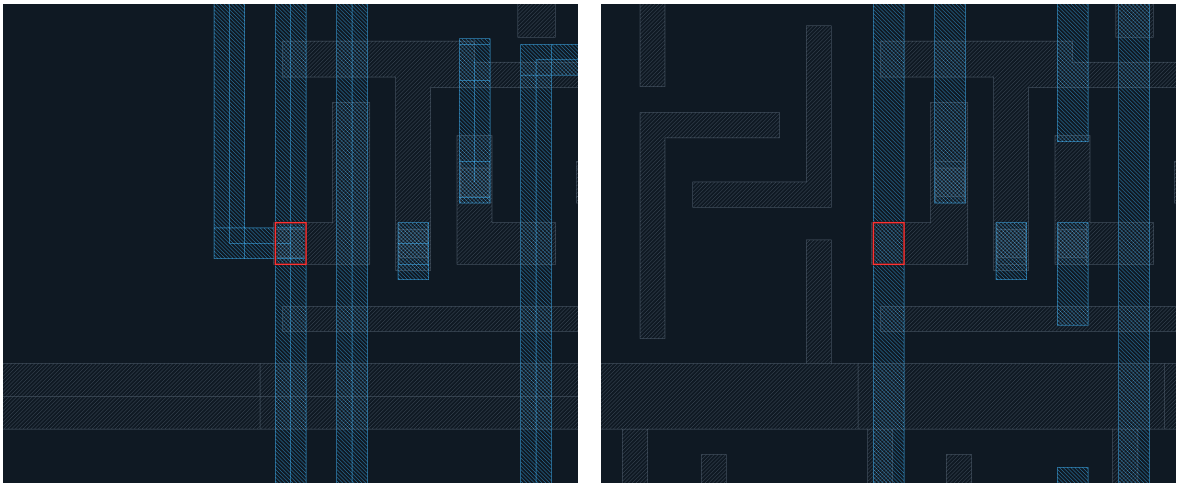
Con US$ 23.000 por mm², el costo equivalente del die final es 0,41445 mm² × 23.000 = US$ 9.532. La aproximación ignora máscaras, rendimiento, encapsulado, pruebas, volumen y pads. La decisión con más impacto en área y costo fue la memoria: al 30 %, el die de B es 3,9 veces el de A (US$ 49.652 contra US$ 12.592).

6. SIGNOFF Y LAYOUT

Resultados finales de final_A_v2_p100_u40: DRC de Magic con 0 violaciones, LVS de Netgen sin diferencias y 0 violaciones de antena. Durante el flujo, OpenROAD detectó 4 redes con violación de antena en Metal2 (la peor, 1,57 veces el límite); LibreLane insertó 2 diodos, volvió a rutear y quedaron en 0.

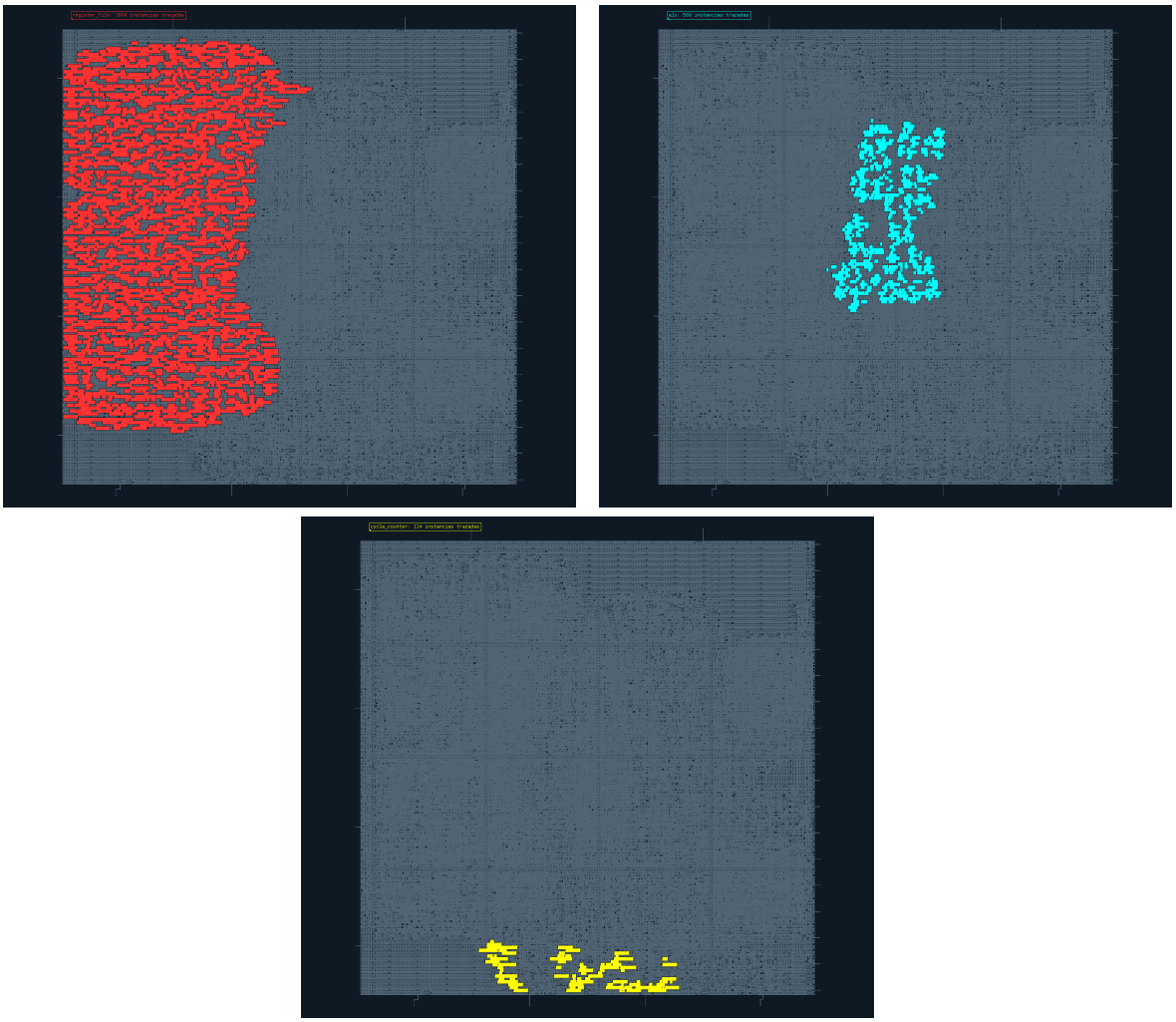
Violación resuelta: en la primera pasada del router hubo 1.924 violaciones de DRC. Una de ellas fue un corto en Metal2 entre la red regs[2][11] del register file y net462, en (33,74; 154,09)–(34,02; 154,47) µm. Para capturarla se repitió el routing deteniéndolo en esa primera pasada (DRT_OPT_ITERS = 0). Las iteraciones de reparación del router la eliminaron (1.924, 665, 476, 3 y 0 violaciones), como muestra la Figura 2.

Figura 2: Corto en Metal2 en la primera pasada del router (izquierda) y la misma zona en el GDS final, sin violaciones (derecha)



Para ubicar los bloques se usaron los nombres jerárquicos conservados (u_rf, u_alu y u_cycle_counter): scripts/extract_layout.py obtiene sus celdas desde la base de datos de OpenROAD y se marcaron sobre el GDS final en KLayout (Figura 3).

Figura 3: Register file (rojo), ALU (celeste) y contador de ciclos (amarillo) en el layout final



7. SIMULACIÓN FUNCIONAL POST-SÍNTESIS

Se simularon con Icarus Verilog los netlists post-síntesis de A y B, mapeados a celdas GF180 y sin retardos anotados (sin SDF), con el programa del juego y el testbench sim/asic_game_tb.v. Este presiona los botones en ciclos fijos: el correcto 127 ciclos después de encenderse el LED y, en la ronda 2, uno incorrecto. Para acortar la simulación, las tres constantes de tiempo se reducen a 300, 100 y 1.000 ciclos, igual en la simulación RTL y en la de los netlists, que se sintetizaron con ese programa (scripts/synth_hierarchy.py). Los estímulos son idénticos en todas las simulaciones.

Prueba	**RTL (A y B)**	**Netlist A**	**Netlist B**
Ciclos medidos en una ronda correcta	136	136	136
Pulsación incorrecta en la ronda 2	Reinicia la ronda	Reinicia la ronda	Reinicia la ronda
Total de las diez rondas correctas	1.360	1.360	1.360

Las trazas del RTL y de los netlists son idénticas línea a línea (por ejemplo, results/functional/asic_game_tb_0.log y results/synthesis/A_sim_hier/gate_sim.log). Se miden 136 ciclos y no 127 porque el programa tarda algunos ciclos en leer el botón y el contador.